



UNIVERSIDAD ESTATAL DE MILAGRO

**FACULTAD DE CIENCIAS E INGENIERÍA
CARRERA DE INGENIERÍA DE SOFTWARE**

TEMA:

Diseño de un sistema de reservas de canchas deportivas para un
polideportivo comunitario

AUTORES:

Regalado Ashley, David Jumbo Ponce, Orellana Arias Angel,
Patiño Jorge, Altamirano Rubén.

GRUPO: #5

ASIGNATURA:

Arquitectura y diseño de software

DOCENTE:

ING. MARIA GABRIELA ESPINOZA BRAVO

FECHA DE ENTREGA:

02/06/2026

PERIODO:

Abril 2026 a Agosto 2026

MILAGRO-ECUADOR

INDICE:

1) INTRODUCCIÓN	3
2) DESARROLLO.....	4
Problema principal	4
Solución propuesta.....	4
Arquitectura del sistema	5
Actores del sistema	7
Requerimientos del Sistema.....	7
Herramientas	10
Cuestiones clave del diseño de software.....	11
Concurrencia	11
Persistencia de datos	12
Manejo de errores y excepciones	12
Seguridad del sistema	13
3) DIAGRAMAS UML	15
Diagrama de Arquitectura MVT	15
Diagrama de clases	17
Diagrama de secuencia	18
Diagrama de Casos de Uso	22
4) CONCLUSION	24
5) REFERENCIAS:.....	25

CASO DE ESTUDIO:

Diseño de un sistema de reservas de canchas deportivas para un polideportivo comunitario

1) INTRODUCCIÓN

En la actualidad, la gestión de recursos en espacios comunitarios se ha convertido en una necesidad creciente. Muchos polideportivos aún gestionan las reservas de canchas deportivas mediante cuadernos, llamadas telefónicas o mensajes por redes sociales, lo que genera problemas recurrentes como la pérdida de información, la duplicación de horarios y los conflictos entre usuarios. Esta situación evidencia la carencia de herramientas tecnológicas que permitan organizar y optimizar el uso de las instalaciones deportivas de manera eficiente.

Teniendo este contexto, el presente trabajo propone diseñar un sistema de reservas de canchas deportivas para un polideportivo comunitario, de esta manera buscamos organización, reducir errores, conflictos y fomentar un uso más equitativo de las instalaciones deportivas

2) DESARROLLO

Problema principal

El polideportivo de una comunidad gestiona actualmente las reservas de canchas deportivas (fútbol, básquet, vóley, entre otras) mediante cuadernos físicos, llamadas telefónicas o mensajes por redes sociales. Este método manual genera reservas duplicadas, pérdida de información y frecuentes discusiones por horarios. No existe un sistema centralizado que controle en tiempo real la disponibilidad de canchas, registre de forma confiable quién realizó cada reserva ni permita validar el acceso de los usuarios el día de uso.

Solución propuesta

Se propone el diseño de un sistema web de reservas de canchas deportivas para un polideportivo comunitario. Los usuarios de la comunidad podrán consultar la disponibilidad y registrar reservas en línea, mientras que el personal del polideportivo administrará canchas, horarios y reservas desde un panel de gestión.

El sistema controlará la concurrencia mediante transacciones atómicas en la base de datos para evitar reservas dobles en la misma franja horaria. Almacenará de forma persistente la información de usuarios, canchas y reservas en una base de datos relacional. Manejará errores con mensajes claros al usuario final y registrará logs internos para auditoría. Aplicará mecanismos de seguridad basados en autenticación por usuario y contraseña, roles de autorización y validación de datos de entrada.

Al confirmarse una reserva, el sistema generará un código QR único que el usuario presentará al llegar al polideportivo. El personal lo escaneará con un dispositivo móvil para validar

en tiempo real que la reserva corresponde a la fecha y horario correcto y así permitir el acceso a la cancha.

Además, el sistema implementa un programa de fidelización por puntos para incentivar el uso ordenado de las canchas. Cada vez que un usuario realiza y cumple correctamente una reserva (sin ausencias injustificadas), el sistema le otorga puntos que se acumulan en su cuenta. Estos puntos pueden canjearse por beneficios como bebidas en el bar comunitario, minutos adicionales de uso cuando la cancha esté libre o prioridad para participar en torneos organizados por el polideportivo.

Arquitectura del sistema

La arquitectura propuesta sigue el patrón Modelo-Vista-Template (MVT), propio del framework Django, organizado en tres capas lógicas que separan responsabilidades y facilitan el mantenimiento, la escalabilidad y la claridad estructural del sistema. Esta arquitectura permite distribuir adecuadamente la gestión de datos, la lógica de negocio y la presentación de la información en la aplicación web.

Capa de presentación (Template)

Es la capa encargada de la interfaz de usuario y de la visualización de la información dentro del sistema. Está compuesta por las páginas web que muestran la disponibilidad de canchas, los formularios de registro e inicio de sesión, el historial de reservas del usuario, los paneles de administración y la interfaz de escaneo del código QR para el personal del polideportivo.

En Django, esta capa se implementa mediante Templates, utilizando HTML, CSS y, si es necesario, JavaScript para mejorar la interacción del usuario. Su función principal es presentar los

datos procesados por la vista de manera clara, ordenada y accesible para cada tipo de actor del sistema.

Capa de lógica de negocio (Vista)

Es la capa donde se encuentra la lógica funcional del sistema. Aquí se procesan las solicitudes del usuario, se validan los datos enviados desde los formularios, se verifica la disponibilidad de las canchas, se controla la concurrencia mediante transacciones atómicas, se calculan los puntos de fidelización, se generan los códigos QR y se aplican las restricciones de acceso según el rol del usuario.

En Django, esta capa corresponde a las Views, que reciben las peticiones HTTP, se comunican con el modelo para consultar o actualizar datos, y finalmente envían la respuesta al template correspondiente. De esta manera, las vistas actúan como el componente central que coordina el comportamiento del sistema.

Capa de acceso a datos (Modelo)

Es la capa responsable de la gestión y persistencia de los datos en la base de datos relacional MySQL. Aquí se definen las entidades principales del sistema, como Usuario, Cancha, HorarioDisponible, Reserva, PuntosHistorial y LogErrores, además de las relaciones entre ellas y las operaciones CRUD necesarias para su administración.

En Django, esta capa se implementa mediante los Models, que permiten representar las tablas de la base de datos como clases de Python. A través del ORM de Django, el sistema puede

consultar, insertar, actualizar y eliminar datos sin necesidad de escribir SQL manualmente, lo que mejora la mantenibilidad, la seguridad y la portabilidad del sistema.

Actores del sistema

Usuario: Consulta disponibilidad de canchas, realiza reservas gratuitas, puede cancelarlas, acumula puntos por uso responsable y presenta el código QR al ingresar al polideportivo.

Personal recepcionista: Registra reservas presenciales o telefónicas en el sistema, consulta el calendario diario y semanal, escanea el código QR con el teléfono para validar el acceso y puede consultar el historial de uso.

Administrador: Crea y actualiza canchas, define horarios disponibles, administra usuarios y roles, configura las reglas del programa de puntos y consulta reportes de uso y estadísticas.

Municipio: Como propietario y gestor del polideportivo, el Municipio es un actor directo que utiliza el panel administrativo del sistema para gestionar la infraestructura, configurar los horarios disponibles, establecer las reglas del programa de puntos y extraer reportes estadísticos operativos.

Requerimientos del Sistema

Requerimientos Funcionales

ID	Requerimiento	Descripción
RF-01	Registro de usuarios	El sistema permite registrar usuarios con nombre, correo, contraseña y rol asignado
RF-02	Inicio de sesión seguro	Los usuarios ingresan con usuario y contraseña; el acceso se controla por roles

ID	Requerimiento	Descripción
RF-03	Consulta de disponibilidad	El usuario puede ver las canchas y franjas horarias disponibles por fecha
RF-04	Realizar reserva	El usuario selecciona cancha y horario; el sistema confirma mediante transacción atómica
RF-05	Cancelar reserva	El usuario puede cancelar una reserva activa desde su cuenta
RF-06	Registro de reserva presencial	El recepcionista puede registrar reservas en nombre de un usuario
RF-07	Generación de código QR	Al confirmar la reserva, el sistema genera un código QR único e irrepetible
RF-08	Validación de acceso por QR	El recepcionista escanea el QR para verificar fecha, hora, token y estado de la reserva
RF-09	Gestión de canchas y horarios	El administrador crea, edita y desactiva canchas y sus franjas horarias disponibles
RF-10	Administración de usuarios y roles	El administrador gestiona cuentas, roles y permisos del personal
RF-11	Programa de puntos	El sistema otorga puntos por asistencia y permite su canje por beneficios
RF-12	Consulta de reportes estadísticos y	El administrador y el Municipio pueden extraer reportes de uso del polideportivo

Requerimientos No Funcionales

ID	Requerimiento	Descripción
RNF-01	Seguridad en el acceso	Contraseñas cifradas con PBKDF2; tokens QR firmados criptográficamente; protección CSRF
RNF-02	Autorización por roles	Tres niveles de acceso: Usuario, Personal y Administrador, con permisos diferenciados
RNF-03	Integridad de datos	Restricción <code>UNIQUE</code> en la BD sobre (idCancha, fecha, horaInicio, horaFin) para evitar duplicados
RNF-04	Disponibilidad del sistema	Plataforma web accesible desde cualquier navegador en dispositivo móvil o escritorio
RNF-05	Tiempo de respuesta eficiente	La consulta de disponibilidad y el registro de reservas deben ejecutarse con respuesta inmediata
RNF-06	Mantenibilidad	Arquitectura MVC que permite modificar cada capa de forma independiente sin afectar las demás
RNF-07	Escalabilidad	El diseño permite incorporar futuras funcionalidades como app móvil o notificaciones
RNF-08	Trazabilidad y auditoría	Todos los errores se registran en la tabla <code>log_errores</code> con fecha, módulo y nivel de severidad

Herramientas

Backend (Servidor)

- Python — lenguaje de programación principal del sistema
- Django — Framework web de Python que implementa el patrón MVT (Model-View-Template). Gestiona el enrutamiento de URLs, las vistas (controladores de lógica), las plantillas HTML, el middleware, el ORM para la base de datos y la protección de seguridad CSRF.

Base de datos

- MySQL — base de datos relacional donde se almacenan todas las tablas (usuarios, canchas, reservas, puntos, log_errores)
- ORM de Django — permite comunicarse con MySQL sin escribir SQL directamente, usando código Python

Frontend:

- Django Templates — para construir las páginas web del usuario, el panel de administración y la interfaz de escaneo QR

Seguridad

- bcrypt / PBKDF2 — algoritmos para encriptar contraseña
- UUID / tokens criptográficos — para generar los códigos QR únicos e irrepetibles

Cuestiones clave del diseño de software

Concurrencia

El sistema debe evitar que dos usuarios reserven la misma cancha en la misma franja horaria simultáneamente. Este es un problema clásico de concurrencia en sistemas de reservas.

Para resolverlo, la operación de crear una reserva se ejecuta dentro de una transacción atómica en la base de datos. Dentro de esa transacción, antes de guardar la nueva reserva, el sistema verifica si ya existe una reserva confirmada para la misma combinación de cancha, fecha y franja horaria (horaInicio, horaFin). Si existe, rechaza la nueva reserva y muestra un mensaje claro al usuario: "La cancha ya está reservada en ese horario. Por favor seleccione otro horario disponible."

Como mecanismo adicional de protección, se define una restricción UNIQUE a nivel de base de datos sobre la combinación (idCancha, fecha, horaInicio, horaFin). De esta forma, si dos peticiones simultáneas pasan la verificación lógica al mismo tiempo, solo una logrará insertar el registro; la otra generará una excepción de violación de restricción que será capturada por la capa de aplicación para informar al usuario de manera amigable.

Persistencia de datos

Toda la información del sistema se almacena en una base de datos relacional (MySQL), asegurando que los datos no se pierdan y que se puedan consultar registros históricos en cualquier momento.

Las tablas principales del sistema son las siguientes.

- La tabla usuarios almacena id, nombre, correo electrónico, teléfono, contraseña, rol y puntos acumulados.
- La tabla canchas almacena id, nombre, tipo de deporte, ubicación dentro del polideportivo y estado (activa/inactiva).
- La tabla horarios_disponibles almacena id, idCancha, día de la semana, horaInicio y horaFin.
- La tabla reservas almacena id, idUsuario, idCancha, fecha, horaInicio, horaFin, estado (confirmada, cancelada, noAsistida) y códigoQR.
- La tabla puntos_historial almacena id, idUsuario, idReserva, puntos otorgados, tipo de movimiento (acumulación o canje) y fecha.
- La tabla log_errores almacena id, fecha, descripción del error, módulo de origen y nivel de severidad.

Manejo de errores y excepciones

El sistema debe ser robusto ante errores tanto de uso como técnicos, sin interrumpir la aplicación ni exponer información sensible al usuario final.

En cuanto a las validaciones, el sistema no permite reservas en horarios fuera del horario del polideportivo, no permite que un usuario tenga más reservas simultáneas de las permitidas

según la configuración del administrador, y no otorga puntos si la reserva se marca como no asistida.

Respecto a los errores técnicos, el sistema maneja las siguientes situaciones. Cuando un usuario intenta reservar una cancha ya ocupada, se captura la excepción de violación de restricción única y se devuelve un mensaje claro al usuario indicando que seleccione otro horario. Si ocurre un fallo de conexión con la base de datos, el error se registra en la tabla `log_errores` con la fecha, el módulo de origen y la descripción, y se muestra al usuario un mensaje genérico: "Ocurrió un problema al procesar su solicitud. Por favor, inténtelo más tarde." Si se escanea un código QR inválido o expirado, el sistema verifica que el token del QR exista en la base de datos, que corresponda a la fecha y hora actuales y que la reserva tenga estado "confirmada"; si alguna de estas condiciones no se cumple, se rechaza el acceso y se notifica al personal.

El manejo de excepciones se centraliza en la capa de aplicación mediante middleware global (por ejemplo, en Django, usando un middleware personalizado de excepciones o decoradores `try/except` en las vistas). Esto permite registrar cada error de forma uniforme y devolver respuestas consistentes en formato JSON con códigos HTTP apropiados.

Seguridad del sistema

Aunque el uso de las canchas es gratuito, el sistema maneja datos personales y debe garantizar la confidencialidad, integridad y disponibilidad de la información.

En cuanto a la autenticación, cada usuario de la comunidad tiene una cuenta con nombre de usuario y contraseña. Las contraseñas se almacenan utilizando un algoritmo para encriptar como

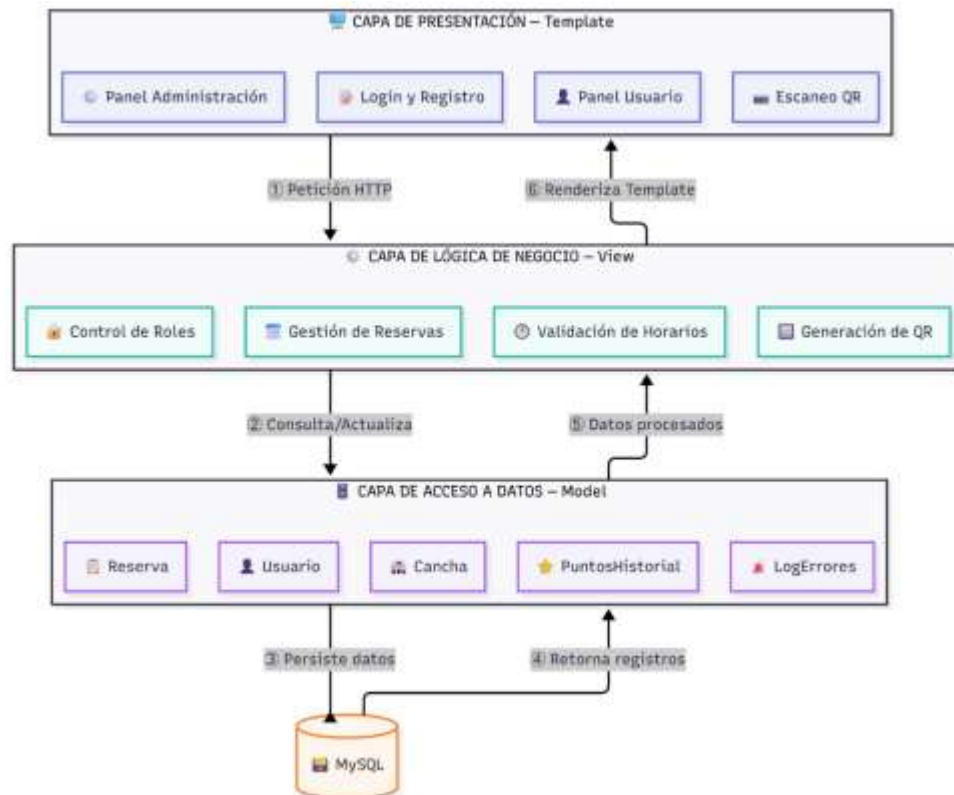
PBKDF2 lo que garantiza que incluso si la base de datos se viera comprometida, las contraseñas originales no serían recuperables.

En cuanto a la autorización basada en roles, el sistema define tres niveles. El rol "Usuario" permite gestionar únicamente sus propias reservas, consultar sus puntos y canjear beneficios. El rol "Personal" permite ver y gestionar todas las reservas, escanear códigos QR y marcar reservas como no asistidas. El rol "Administrador" incluye además la gestión de canchas, horarios, usuarios y las reglas del programa de puntos.

En cuanto a la seguridad del código QR, el token contenido en el QR no es el identificador simple de la reserva, sino un UUID (Universally Unique Identifier) o una cadena aleatoria firmada criptográficamente que solo el sistema puede validar. Al escanearlo con el teléfono, se abre una URL de validación protegida que comprueba el token, la fecha, la hora y el estado de la reserva antes de autorizar el acceso. Los tokens tienen una vigencia limitada a la fecha y franja horaria de la reserva, lo que impide su reutilización.

3) DIAGRAMAS UML

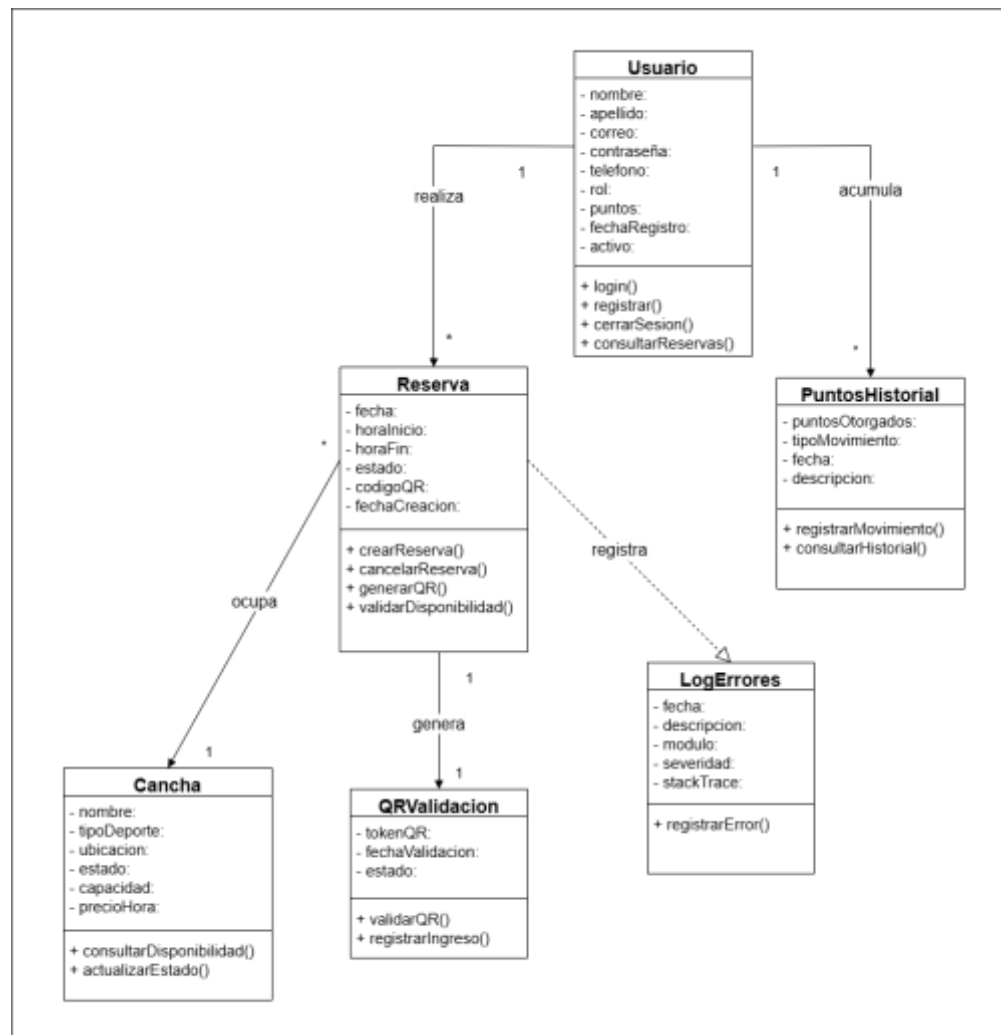
Diagrama de Arquitectura MVT



El diagrama representa la organización estructural del sistema web bajo el patrón Modelo-Vista-Template (MVT) de Django, distribuido en tres capas lógicas independientes. La capa Template constituye la interfaz de usuario y contiene las pantallas de Login y Registro, el Panel del Usuario, el Panel de Administración y la interfaz de Escaneo QR; esta capa únicamente presenta información sin contener lógica de negocio. La capa View es el núcleo funcional del sistema y se encarga de procesar las solicitudes HTTP, validar horarios mediante transacciones

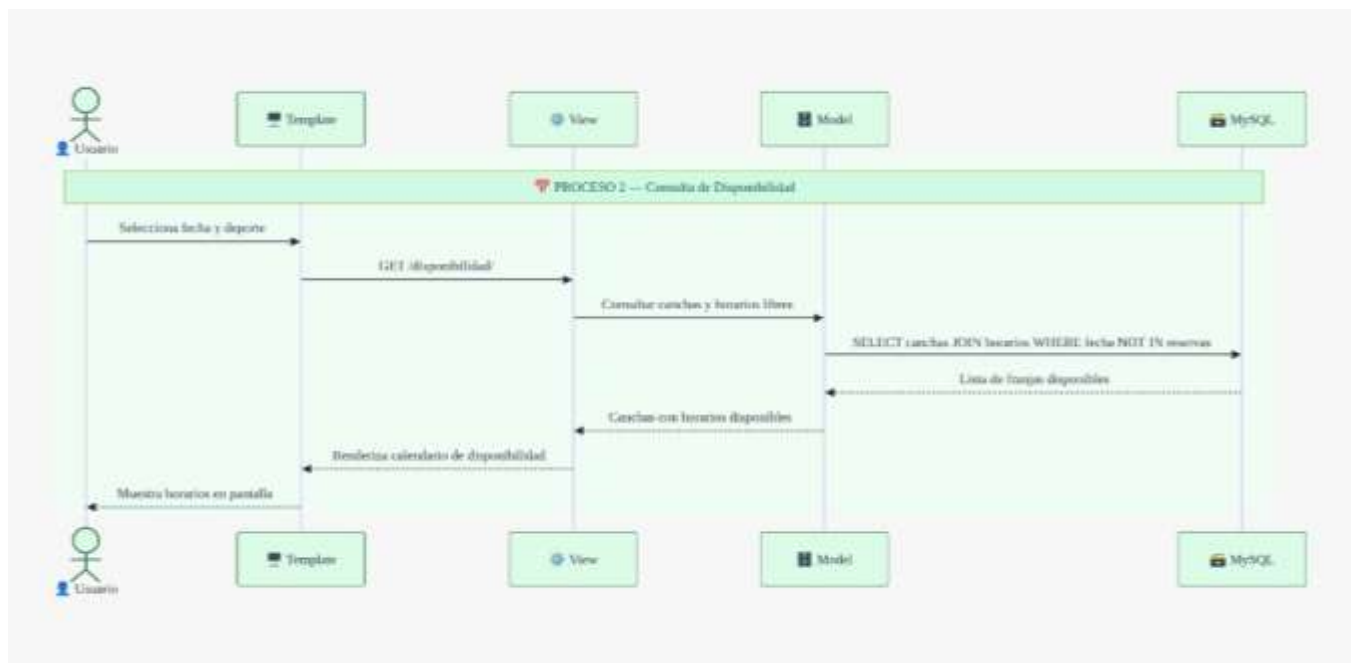
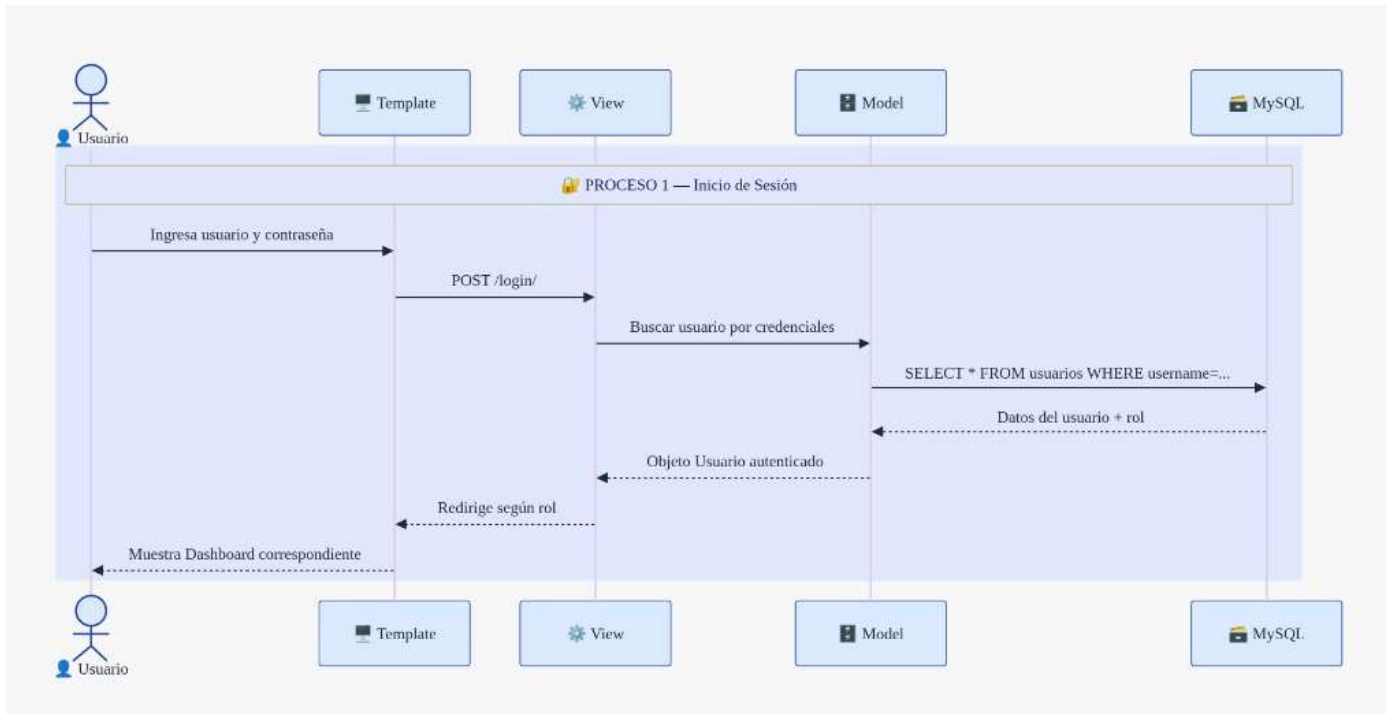
atómicas, aplicar el control de roles, gestionar las reservas y generar los tokens criptográficos de los códigos QR. La capa Model define las cinco entidades principales del sistema —Usuario, Cancha, Reserva, PuntosHistorial y LogErrores— y gestiona toda la comunicación con la base de datos relacional MySQL a través del ORM de Django, sin necesidad de escribir SQL directamente. El flujo de comunicación sigue una secuencia de seis pasos: el usuario envía una petición desde el Template, la View la procesa y consulta al Model, el Model persiste o recupera los datos en MySQL, y finalmente la View renderiza el Template con la respuesta correspondiente.

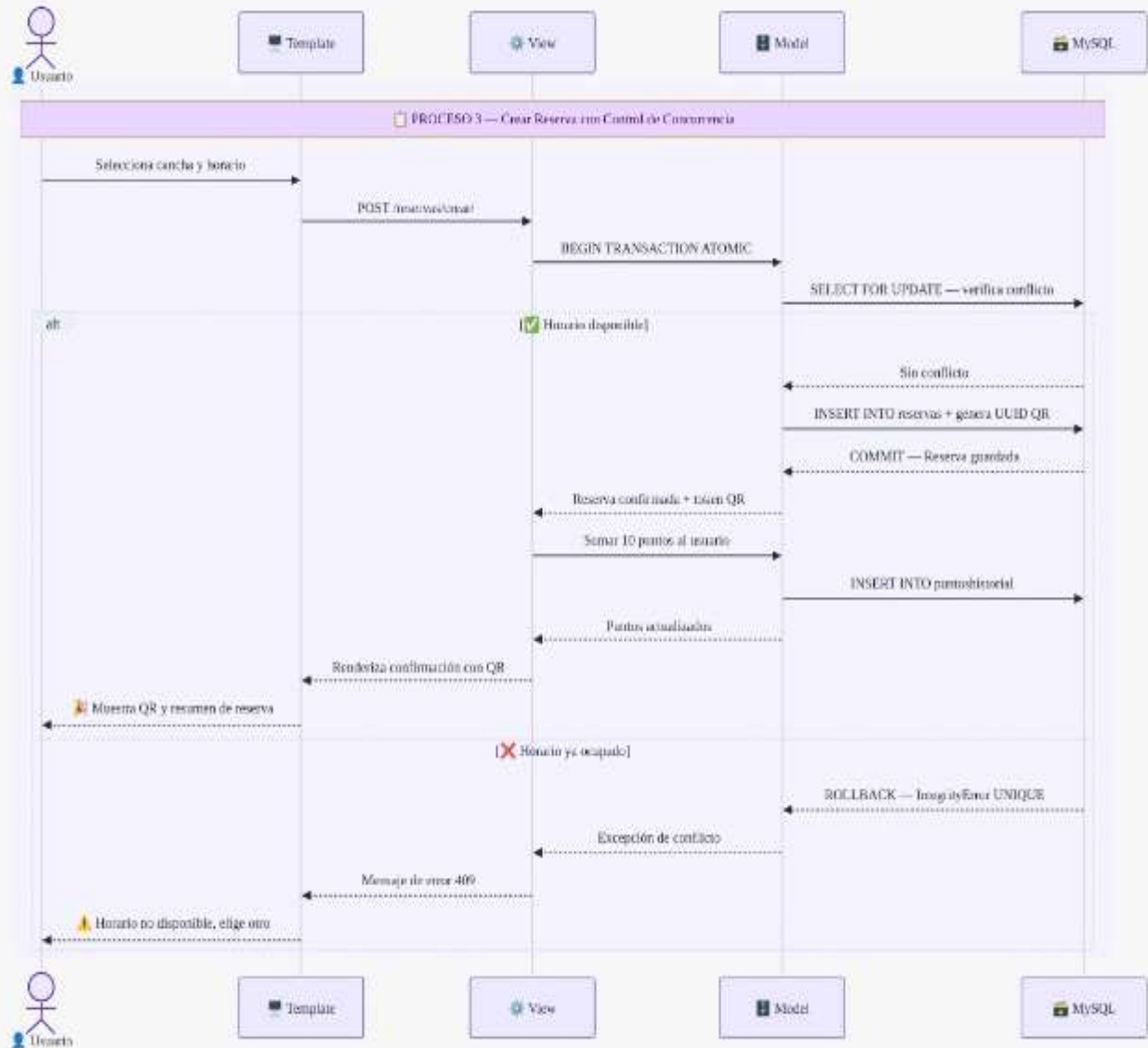
Diagrama de clases



El diagrama modela las seis entidades principales del sistema: Usuario, Reserva, Cancha, QRValidacion, PuntosHistorial y LogErrores, junto con sus atributos, métodos y relaciones. Un usuario puede realizar múltiples reservas y acumular puntos; cada reserva ocupa una cancha y genera un registro de validación QR; los errores del sistema quedan registrados en LogErrores para auditoría.

Diagrama de secuencia



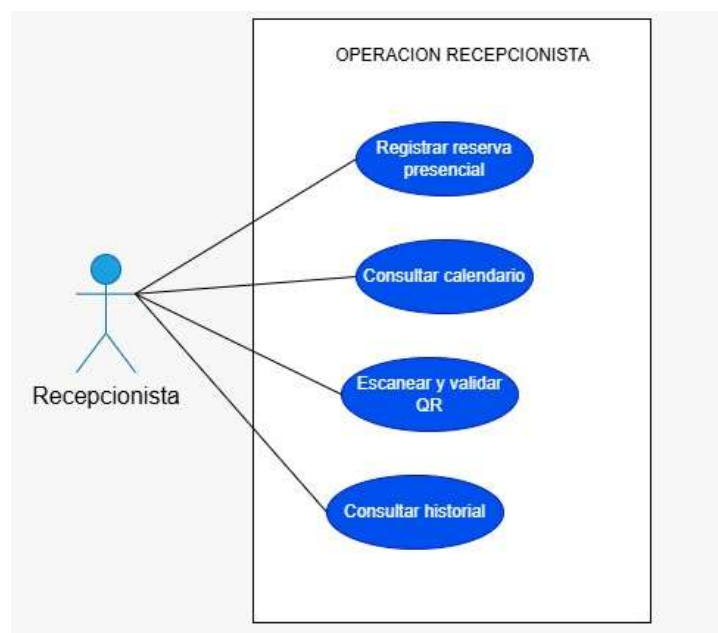
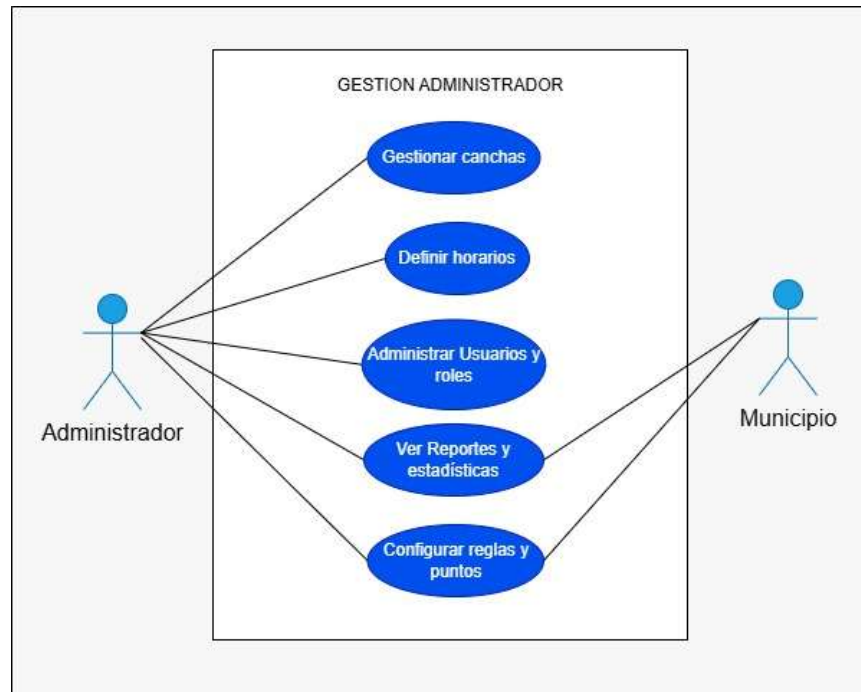


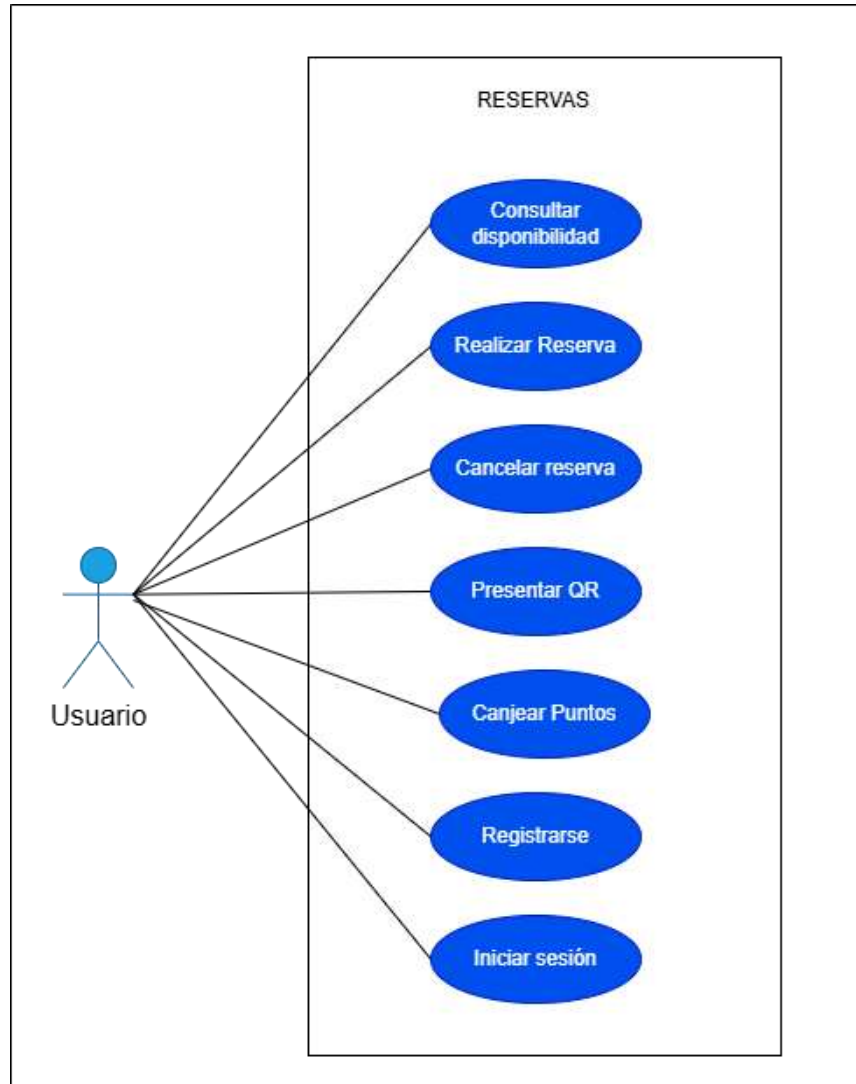


El diagrama de secuencia ilustra la interacción entre los cinco componentes del sistema bajo el patrón MVT de Django —Usuario, Template, View, Model y MySQL— a lo largo de cuatro procesos clave. En el Proceso 1 (Inicio de Sesión), el usuario ingresa sus credenciales desde el Template, la View autentica los datos consultando al Model, este ejecuta la consulta en MySQL y retorna el objeto de usuario con su rol, redirigiendo la respuesta al dashboard correspondiente.

En el Proceso 2 (Consulta de Disponibilidad), el usuario selecciona fecha y deporte, la View solicita al Model las franjas horarias libres mediante una consulta JOIN en MySQL, y el resultado se renderiza en el Template como un calendario de disponibilidad. En el Proceso 3 (Crear Reserva), la operación más crítica del sistema, la View ejecuta una transacción atómica con bloqueo SELECT FOR UPDATE para verificar conflictos de concurrencia; si el horario está disponible se realiza el INSERT de la reserva junto con la generación del UUID del código QR y se acumulan 10 puntos al usuario, mientras que si el horario ya está ocupado se ejecuta un ROLLBACK y se informa al usuario con un mensaje de error 409. Finalmente, en el Proceso 4 (Validación QR), el personal escanea el código QR, la View busca el token UUID en la base de datos verificando fecha, hora y estado de la reserva; si es válido se autoriza el acceso, y si es inválido o expirado se registra el evento en la tabla LogErrores y se deniega el ingreso.

Diagrama de Casos de Uso





Justificación del modelo

El modelo se estructuró con clases que representan las entidades y procesos principales del sistema, asegurando coherencia entre los diagramas y los requerimientos. Reserva concentra la lógica central del negocio, mientras que Usuario, Cancha, PuntosHistorial, QRValidacion y LogErrores permiten modelar el comportamiento completo del sistema de reservas, validación y control. Esta organización facilita el mantenimiento, la trazabilidad y la correcta interacción entre la vista, el controlador y el modelo.

4) CONCLUSION

El diseño de este sistema de reservas de canchas deportivas representa una solución integral frente a los métodos manuales de gestión que tradicionalmente se emplean en polideportivos comunitarios. A lo largo del trabajo se abordaron las cuatro cuestiones clave del diseño de software establecidas: la concurrencia se resuelve mediante transacciones atómicas y restricciones de unicidad en la base de datos; la persistencia se garantiza con un modelo relacional bien estructurado en MySQL; el manejo de errores se centraliza mediante middleware con mensajes claros para el usuario y registros internos en la tabla LogErrores para auditoría; y la seguridad se implementa a través de autenticación con hash PBKDF2, autorización basada en roles y tokens criptográficos UUID para los códigos QR.

La arquitectura MVT (Modelo-Vista-Template) por capas, propia del framework Django, permite que cada componente del sistema pueda evolucionar de forma independiente: el Model gestiona las entidades y la persistencia de datos, la View concentra toda la lógica de negocio y el procesamiento de solicitudes, y el Template se encarga exclusivamente de la presentación de la información al usuario. Esta separación clara de responsabilidades facilita el mantenimiento futuro y la incorporación de nuevas funcionalidades como notificaciones por correo electrónico, una aplicación móvil nativa o la integración con sistemas de pago si el polideportivo decide cobrar por el uso de las canchas.

5) REFERENCIAS:

Pressman, R. S., & Maxim, B. R. (2014). Ingeniería de software: Un enfoque práctico (8.^a ed.). McGraw-Hill.

Sommerville, I. (2011). Ingeniería de software (9.^a ed.). Addison-Wesley.

Bourque, P., & Fairley, R. E. (Eds.). (2014). SWEBOK v3.0: Guide to the Software Engineering Body of Knowledge. IEEE Computer Society.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.